

Isabella Mastroeni

LINGUAGGI

Cap. 3 ESPRESSIONI

```
5
(ast):
e = getNodeName()
symbol.sym_name.get(int(ast[0]), ast
%s [label="%s" % (nodename, la
instance(ast[1], str):
ast[1].strip():
print '%s' % ast[1]
else:
print ""

print "";
children = []
for n, child in enumerate(ast[1:]):
children.append(dotwrite(child,
%s -> { % nodename,
print
for name in children:
print '%s' % name,
```

ESPRESSIONI

- Cosa sono?
- A cosa servono?
- Cosa rappresentano?

- Abbiamo già detto che il significato di una espressione è essenzialmente un valore, potenzialmente con dei vincoli (ad esempio il tipo) ma comunque un valore. Per manipolare valori dobbiamo necessariamente rappresentarli e per questo usiamo le espressioni.
- Le espressioni devono essere valutate per dare un valore, e va osservato che il concetto di valore è diverso da quello di locazione di memoria: i valori sono parte dello specifico PL (PL diversi possono manipolare valori diversi), la locazione è parte della struttura dell'architettura. Ovvero la locazione non è parte del PL ma modella il modo con cui il PL si relaziona alla macchina sottostante.
- Quindi le espressioni sono oggetti sintattici utilizzati esclusivamente per ragionare sui significati che rappresentano. Quindi la sintassi è solo lo strumento per ragionare sul significato, come le lingue sono solo lo strumento per comunicare gli stessi significati in paesi diversi.

DESCRIVERE LE ESPRESSIONI

Abbiamo detto che le espressioni sono oggetti sintattici usati per ragionare sui loro significati, che sono i valori. Per associare significati/valori alle espressioni, le espressioni devono essere VALUTATE.

Il valore ottenuto mediante la valutazione di una espressione è detto valore esprimibile, in quanto è un valore che può essere espresso attraverso la sintassi del PL, e in particolare attraverso la sintassi delle espressioni.

VALORI ESPRIMIBILI. $Eval = bool \cup int$ con metavariable ev

In generale, in un linguaggio come PL0, abbiamo espressioni che rappresentano interi e identificatori, e indirettamente anche booleani in quanto le condizioni corrispondono a valori booleani ($3=3$ è true, mentre $3\#3$ è false).

Quindi i valori esprimibili mediante espressioni sono entrambi gli insiemi di valori.

Dal punto di vista sintattico, i costituenti elementari delle espressioni sono i letterali (costanti e identificatori), i quali vengono composti mediante operatori. Tali operatori, ovviamente possono essere sia operatori aritmetici che booleani.

Riassumendo, una espressione è un oggetto sintattico composto da operatori, operandi (che sono a loro volta altre espressioni), parentesi e, potenzialmente, chiamate a funzioni/procedure (questo lo vedremo più avanti).

COME SI CARATTERIZZANO

Entriamo ora nel merito di come possono essere fatte le espressioni e di quali aspetti le caratterizzano. Abbiamo già visto che possiamo descrivere la grammatica delle espressioni in modo che questi aspetti siano almeno in parte risolti dalle regole grammaticali, anche se per dare poi semantica useremo una grammatica potenzialmente più semplice. Vediamo nel dettaglio le caratteristiche più importanti:

- Notazione
 - La notazione specifica in che modo gli operandi e gli operatori vengono rappresentati e per indicare su quali operandi un operatore opera. Come vedremo abbiamo tre notazioni possibili: infissa, prefissa e postfissa.
- Regole di precedenza (tra operatori)
- Regole di associatività (degli operatori)

- Ordine di valutazione degli operandi
- Presenza di side-effects
 - Per la valutazione delle espressioni un side-effect è qualunque effetto che non consiste puramente nella rappresentazione di valori
- Overloading degli operatori
 - Nei PL possono esistere simboli di operatore che hanno significato diverso a seconda del tipo degli operatori a cui sono applicati.
- Espressioni con tipi misti
 - In certi PL possiamo avere espressioni che combinano tipi diversi di valori

NOTAZIONE

Le espressioni aritmetiche sono una rappresentazione familiare, fortemente basata su strumenti matematici elementari, ma anche in casi semplici facciamo assunzioni su vari aspetti che la caratterizzano (ad esempio sulle precedenze degli operatori). Tali assunzioni possono dipendere da vari aspetti e tolgono ambiguità alla grammatica specificando un aspetto del controllo relativo alla valutazione delle espressioni. A seconda di come si rappresenta l'espressione varia il modo con cui si determina la semantica e di conseguenza può variare la sua valutazione

Le espressioni possono essere denotate in modi diversi:

- Notazione in-fissa: $a + b$
- Notazione pre-fissa (polacca): $+ a b$
- Notazione post-fissa (polacca inversa): $a b +$

La scelta della notazione può determinare le regole di associatività e/o precedenza, o incidere su altre caratteristiche delle espressioni.

La più usata nelle espressioni aritmetiche è quella infissa perché di più facile ed intuitiva lettura, però è quella in cui ho bisogno di specificare altre regole (associatività e precedenza) o strumenti (le parentesi) per eliminare ambiguità. Infatti, nelle altre notazioni, l'arietà dell'operatore (ovvero il numero di operandi a cui si applica) è spesso sufficiente ad evitare ambiguità nella valutazione.

NOTAZIONE POST-FISSA

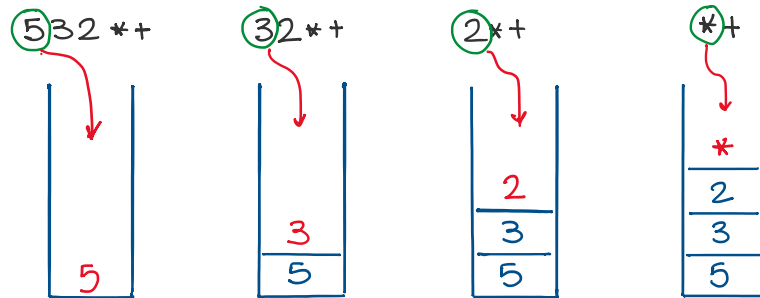
La notazione post-fissa è più semplice dell'in-fissa, dal punto di vista computazionale, in quanto non necessita di regole di precedenza e associatività, e non necessita nemmeno di parentesi. Inoltre, permette una rappresentazione uniforme per qualunque arietà di operatori. Questa notazione è spesso usata nel codice intermedio di alcuni compilatori.

Un esempio di espressione in tale notazione è $ab+cd+*$. La valutazione evolve da sinistra a destra usando una pila LIFO. Per valutare gli operandi bisogna conoscerne il numero dipendentemente dall'operatore, ovvero devo sapere quanti operatori devo leggere (ovvero estrarre dalla pila) quando incontro un operatore. Ad esempio, supponiamo di voler valutare $532*+$, man mano che si legge si mettono i valori sulla pila, quando incontro un operatore (in questo caso $*$), sapendo che la sua arietà è 2, sappiamo di dover applicare l'operatore ai primi due valori in cima alla pila, il risultato viene poi caricato in cima alla pila e si continua.

ALGORITMO DI VALUTAZIONE:

1. Leggi il prossimo simbolo dell'espressione
 - A. Se il simbolo letto è un operatore (di varietà n):
 - Applica l'operatore a n operandi immediatamente precedenti sulla pila,
 - Memorizza il risultato in R ,
 - Elimina operatore ed operandi dalla pila
 - Memorizza il valore di R sulla pila.
 - B. Se il simbolo letto è un operando inseriscilo nella pila.
2. Se ci sono ancora simboli nell'espressione da leggere torna a 1 altrimenti termina (il risultato della valutazione è il valore in cima alla pila).

ESEMPIO



OPERANDO IN CIMA ARIETÀ = 2



NOTAZIONE PRE-FISSA

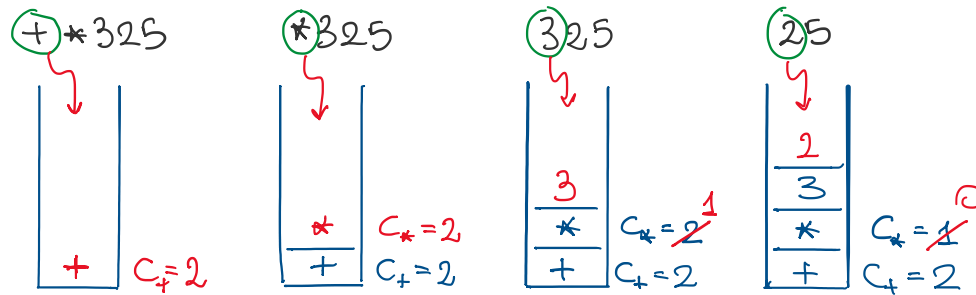
Anche la notazione pre-fissa è più semplice dell'in-fissa, dal punto di vista computazionale, in quanto non necessita di regole di precedenza e associatività, e non necessita nemmeno di parentesi. Inoltre, permette una rappresentazione uniforme per qualunque arietà di operatori. A differenza della notazione post-fissa l'algoritmo di valutazione è più complicato in quanto è necessario contare gli operandi che vengono letti. Questa notazione viene spesso usata per gli operatori unari e per le funzioni definite dall'utente.

Un esempio di espressione in tale notazione è $*+ab+cd$. La valutazione evolve sempre da sinistra verso destra usando una pila LIFO ma in questo caso ogni volta che viene letto un operatore devo contare gli operandi da leggere prima di eseguire il calcolo. Questo significa che dobbiamo usare un contatore, ed in questo senso è più complicato dell'algoritmo precedente.

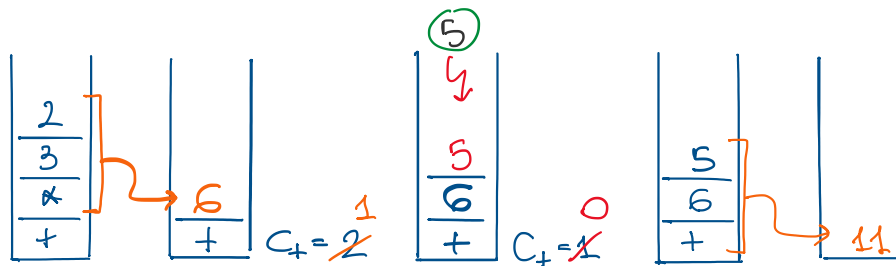
ALGORITMO DI VALUTAZIONE:

1. Leggi il prossimo simbolo dell'espressione e metti nella pila
2. Se il simbolo è un operatore op: $C_{op} = n$ e torna a 1.
3. Se il simbolo letto è un operando inseriscilo nella pila e calcola $C_{op} = C_{op} - 1$ (op ultimo operatore inserito)
4. Se $C_{op} \neq 0$ torna a 1 (op ultimo operatore inserito).
5. Se $C_{op} = 0$ allora
 - Applica l'ultimo operatore op agli operandi in cima alla pila (che vanno cancellati) e carica il risultato nella pila e se non ci sono più simboli vai a 6.
 - $C_{op} = n - m$ (n arietà del nuovo ultimo operatore op nella pila, m numero di operandi sopra l'operatore nella pila)
 - Torna a 4.
6. Se la pila non è vuota torna a 1.

ESEMPIO



C = 0 DELL' OPERATORE IN CIMA -> ESEGNO



NOTAZIONE IN-FISSA

La notazione in-fissa è quella più usata in matematica e di conseguenza nei linguaggi di programmazione almeno come zucchero sintattico. Di fatto è la notazione più chiara ma anche quella che maggiormente si presta ad ambiguità come abbiamo già visto.

La notazione in-fissa infatti, è quella che permette una lettura più intuitiva ma che richiede maggiori specifiche. Per prima cosa è necessario stabilire le regole di precedenza e associatività. Inoltre sono necessario l'utilizzo delle parentesi per permettere precedenze o associatività diverse da quelle stabilite dalle regole, o comunque per aumentare la leggibilità.

Ad esempio, consideriamo $15-5-3$, a seconda dell'operazione che eseguiamo per prima cambia il risultato. La matematica ci dice di andare da sinistra verso destra, ottenendo 7, ma con le parentesi possiamo permettere eccezioni ed ottenere $15-(5-3)$, ottenendo 13.

In ogni caso, anche in matematica l'associatività non va sempre da sinistra verso destra, un esempio è la potenza. Infine, con la notazione in-fissa non è possibile valutare l'espressione con una sola scansione: Ad esempio per valutare $5+3*2$ è necessario valutare prima $3*2$. Per questo l'algoritmo di valutazione richiede un parsing completo dell'espressione più complesso degli algoritmi visti in precedenza. Gli algoritmi di parsing vengono solitamente visti nel corso di Compilatori.

`a + b * c ** d ** e / f ??`

`if A < B and C < D then ??`
(in Pascal Errore se A,B,C,D non sono tutti booleani)

REGOLE DI PRECEDENZA

Regole di precedenza: Le regole di precedenza di un operatore per la valutazione di una espressione definisce l'ordine in cui operatori "adiacenti" a diversi livelli di precedenza vengono valutati.

Per evitare ambiguità i matematici decisero, tempo fa, di ordinare gli operatori in una gerarchia di priorità di valutazione. Questo permetteva di utilizzare poi questo ordine di valutazione al momento della valutazione dell'espressione matematica. La moltiplicazione ad esempio ha maggiore priorità dell'addizione. Ogni linguaggio può comunque differire per le priorità assegnate agli operatori, la cosa importante è semplicemente stabilire tali priorità. Ad esempio $a+b*c$ cambia risultato in generale a seconda che si valuti da sinistra verso destra o viceversa: $3+4*5$ vale 35 se valutiamo da sinistra, vale invece 23 se (come stabilito dalle regole di precedenza) valutiamo da destra, ovvero prima la moltiplicazione e poi l'addizione.

Tipici livelli di precedenza (mostrati in ordine decrescente)

- Parentesi
- Operatori unari
- ****** (se il linguaggio lo supporta)
- ***, /**
- **+, -**

REGOLE DI ASSOCIATIVITÀ

Regole di associatività: Le regole di associatività per la valutazione delle espressioni definiscono l'ordine con cui operatori allo stesso livello di precedenza vengono valutati.

Abbiamo già visto come può cambiare la valutazione di una espressione a seconda che le operazioni, allo stesso livello, si eseguano da destra verso sinistra o da sinistra verso destra. Il problema si pone ad esempio anche tra operazioni diverse tra le quali non sono fissate regole di precedenza, come tra $+$ e $-$

Tipiche regole di associatività

- Da sinistra verso destra
- A volte operatori unari associano da destra (e.g., in FORTRAN)

- In alcuni linguaggi tutti gli operatori hanno lo stesso livello di precedenza e associano da destra
- Le regole di precedenza e associatività possono essere sovrascritte usando le parentesi

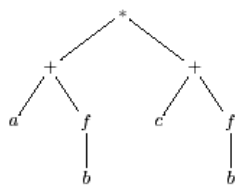
Ad esempio, $a+b-c+d$ restituisce valori diversi a seconda che si associ da destra o da sinistra. In particolare, $15+2-7+9$ restituisce 19 se associamo da destra, oppure 1 se associamo da sinistra.

Concludendo, è fondamentale conoscere quali regole di associatività sono implementate in un linguaggio di programmazione per poterlo usare correttamente ed evitare inattesi errori di valutazione.

VALUTAZIONE DELLE ESPRESSIONI

L'implementazione di un linguaggio di programmazione deve ovviamente rappresentare internamente le espressioni in modo da poterle manipolare.

Tipicamente la rappresentazione interna delle espressioni consiste in una rappresentazione ad albero (*abstract syntax tree*), dove i nodi interni sono operatori mentre le foglie sono valori (ovvero operandi elementari). Ogni nodo interno, etichettato con un operatore, ha per figli gli alberi che rappresentano le espressioni a cui l'operatore si applica. L'albero fornisce quindi la struttura dell'espressione, ovvero elimina eventuali ambiguità legati a precedenze e associatività, ma non può dare informazioni riguardanti l'ordine di valutazione dell'espressione che invece è determinata dalla tipologia di visita che



invece è determinata dalla tipologia di visita che

l'implementazione del linguaggio effettua sull'albero. Va notato che il problema dell'ordine di valutazione non è un problema matematico, in quanto le operazioni sono tutte commutative, ma è un problema informatico, in quanto ci sono vari aspetti (non propriamente matematici) che possono influire sul risultato:

- Operandi non definiti

La presenza di operandi non definiti può far sì che l'espressione risultante possa essere valutata solo con un preciso ordine di valutazione. Si consideri, ad esempio, l'espressione di C $(a == 0 ? b : b/a)$, è chiaro che b/a può essere una divisione per 0, quindi non definita. È anche vero che, per come è scritta l'espressione, serve valutare b/a solo quando a non è 0. Quindi in realtà, valutando le operandi solo quando necessario (valutazione *lazy*) questa espressione è sempre definita. È chiaro quindi che è importante sapere se un linguaggio usa valutazione *lazy* o

eager (ovvero se tutti gli operandi vengono comunque valutati) per poterlo usare correttamente. Nel caso di espressioni booleane, la valutazione *lazy* è spesso detta corto circuito. Altri esempi sono:

`a == 0 || b/a > 2`

Dove b/a potrebbe essere valutato con a uguale a 0

```
p := lista;
while (p != nil) and (p.valore != 3)
do
    p := p.prossimo;
```

Dove $p.valore$ potrebbe essere usato con p uguale a nil.

- Effetti collaterali

Sono effetti non propriamente legati all'oggetto che si sta manipolando (ad esempio se la valutazione di una espressione causa anche un cambiamento della memoria). Si parla di side effect anche quando una funzione modifica i propri parametri o comunque modifica variabili non

locali. Ad esempio, $((a+f(b)) * (c+f(b)))$ con $f(b)$ definita come $b++$, restituisce valori diversi a seconda dell'ordine di valutazione. Per evitare tali problemi alcuni linguaggi (Java) fissano l'ordine di valutazione, altri non permettono la definizione di funzioni con effetti collaterali.

- Aritmetica finita

Nelle macchine l'aritmetica è limitata dall'architettura, quindi i numeri non sono infiniti ed esiste un massimo numero rappresentabile. Il problema di valutazione si ha in caso di espressioni che coinvolgono tale massimo numero, ad esempio se eseguiamo una somma prima di una differenza può cambiare il risultato se la somma calcola un numero maggiore del massimo numero rappresentabile.

Nei linguaggi di programmazione, tipicamente l'ordine di valutazione risolve per prima cosa le variabili, e le costanti poi valuta gli operandi seguendo la struttura data dalle parentesi dell'espressione.

ESPRESSIONI: SEMANTICA

- Quale è il loro significato?
- Cosa significa valutare?
- Come vengono valutate?

- Il significato di una espressione è essenzialmente un valore, potenzialmente con dei vincoli (ad esempio il tipo) ma comunque un valore.
- Le **espressioni** devono essere **valutate** per restituire un valore. I valori sono parte dello specifico linguaggio di programmazione (linguaggi di programmazione diversi possono manipolare valori diversi)
- Due espressioni possono essere *DIVERSE* ma essere valutate nello stesso VALORE (in tutti gli stati): in tal caso sono dette **equivalenti**!
- Perché due espressioni siano equivalenti non basta che denotino lo stesso valore in stati specifici, ma lo devono denotare proprio in tutti gli stati possibili.

SEMANTICA DELLE ESPRESSIONI

Introduciamo ora la semantica delle espressioni di un linguaggio imperativo (basato su architettura di von Neumann), che chiamiamo IMP. Per parlare della semantica non ci interessa la grammatica che include tutte le regole di associatività e di precedenza, ma ci basta quella più semplice anche se ambigua. Di fatto nella semantica si rappresenta, mediante la forma sintattica, il corrispondente albero di valutazione che è invece non ambiguo. Ignoriamo anche le parentesi per semplificare, visto che le regole, essendo induttive, non ne sono influenzate (il caso (E) sarebbe un caso induttivo che toglierebbe solo le parentesi, quindi chiaramente senza rilevanza semantica).

Quindi per parlare di semantica guardiamo alla **struttura induttiva** e non alla struttura sintattica (ignoriamo gli aspetti puramente lessicali). Ad esempio con l'espressione $E + E$ stiamo rappresentando un albero sintattico con come radice $+$ ed i cui sottoalberi (operandi) sono espressioni, quindi generati a loro volta da E .

Ricordiamo che la sintassi che abbiamo introdotto per le espressioni è la seguente:

```
<Exp>  
E → A | B  
A → I | N | A op A  
B → I | true | false | not B | B or B | A = A
```

Dove **A** sono le espressioni aritmetiche in cui **op** sta nell'insieme $\{+, -, *, =\}$, mentre **B** sono le espressioni booleane.

Visto che gli identificatori hanno un significato che coinvolge anche i comandi oltre che i concetti di ambiente e binding, cerchiamo di capire prima le espressioni ignorando gli identificatori, ovvero assumiamo espressioni senza identificatori. Ovvero partiamo a dare significato alla seguente grammatica delle espressioni:

```
<Exp> E → A | B  
A → N | A op A  
B → true | false | not B | B or B | A = A
```


GLI INGREDIENTI

Per prima cosa dobbiamo definire l'insieme $\mathcal{E}$ delle espressioni da valutare ad intero o booleano, ovvero quelli che prima abbiamo detto essere alberi di valutazione, con metavariable e .

Espressioni $\mathcal{E}$:

Insieme di (alberi di valutazione) di espressioni valutate ad intero o booleano: e

Dobbiamo anche definire l'insieme dei valori, ovvero dei numeri della macchina sottostante (con metavariable n, m, p), e dei valori booleani (con metavariable t). È da notare che la macchina sottostante non è necessariamente una macchina fisica, può infatti essere una macchina virtuale. Naturalmente noi non siamo interessati all'architettura di tale macchina, ma solo alle primitive che essa mette a disposizione dei livelli superiori, nel nostro caso le operazioni aritmetiche e gli operatori booleani.

Numeri $\mathcal{N}$:

Insieme di numerali nella macchina sottostante: m, n, p

Booleani $\mathcal{B}$:

Insieme di valori booleani $\{true, false\}$: t

Infine usiamo questi insiemi per definire il sistema di transizione, le cui regole forniranno esattamente la semantica operativa delle espressioni di cui abbiamo dato la grammatica.

Quindi definiamo l'insieme delle configurazioni Γ come l'insieme delle espressioni da valutare $\mathcal{E}$. L'insieme delle configurazioni terminali T sono ovviamente i valori (numerici e booleani). Come sappiamo le configurazioni terminali devono essere un sottoinsieme delle configurazioni, ed effettivamente i valori sono espressioni utate primitive. A partire da questi insiemi possiamo definire il sistema di transizione.

SISTEMA DI TRANSIZIONE:

$\Gamma = \mathcal{E}$, $T = \mathcal{N} \cup \mathcal{B}$, $op \in \{+, -, *, =\}$ $bop \in \{=, or\}$

LE REGOLE DI TRANSIZIONE →

A questo punto introduciamo le regole di transizione. Facciamo attenzione a distinguere tra il simbolo sintattico di una operazione (**bop** in grassetto rosso) e il simbolo dell'operazione nella macchina sottostante (bop normale). Di questo non ci interessa l'implementazione, ma solo che lo possiamo usare per interpretare il simbolo del linguaggio, ovvero eseguiamo astrazione. Partiamo dalle regole necessarie per valutare espressioni aritmetiche.

$$\mathcal{E}_1: m \text{ op } n \rightarrow p \quad \text{se} \quad m \text{ op } n = p, \\ m, n, p \in \mathcal{N}$$

La prima regola è un assioma, che semplicemente valuta l'espressione sintattica contenente un operatore aritmetico, nel valore che esso rappresenta. Ad esempio, $3+5$ è una scrittura sintattica dove il simbolo 3 rappresenta il valore 3, il simbolo 5 rappresenta il simbolo 5, e quindi interpretando il simbolo della somma come addizione restituiamo il simbolo 8 che rappresenta il valore 8.

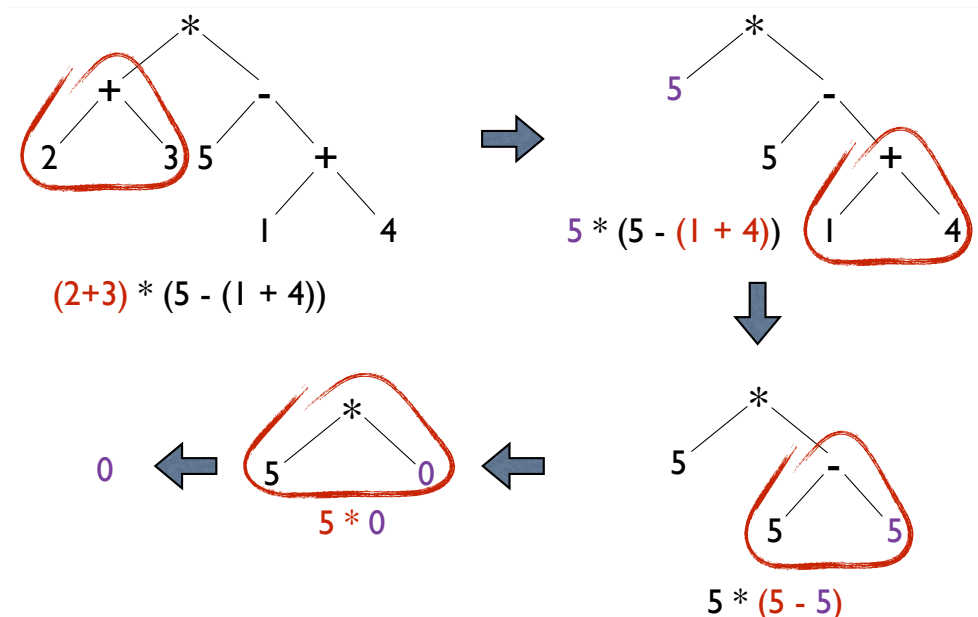
$$\mathcal{E}_2: \frac{e \rightarrow e'}{e \text{ op } e_0 \rightarrow e' \text{ op } e_0}$$

La seconda regola è invece una regola induttiva di valutazione. Questa regola mi dice che, se gli operandi non sono valori primitivi allora dobbiamo valutarli. In particolare, questa regola mi dice che prima valutiamo l'espressione a sinistra dell'operatore. La regola successiva stabilisce invece che, nel momento in cui l'operando a sinistra è un valore allora posso iniziare a valutare l'operando a destra.

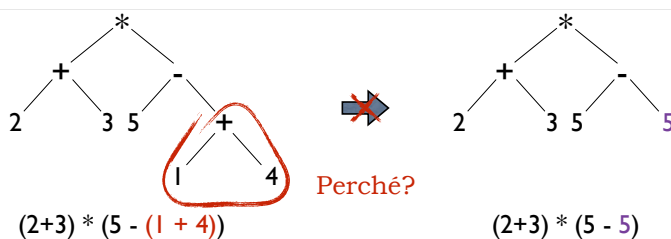
Chiaro che quando anche l'operatore a destra è un valore allora ricadiamo nell'assioma e possiamo restituire il valore finale.

$$\mathcal{E}_3: \frac{e \rightarrow e'}{m \text{ op } e \rightarrow m \text{ op } e'}$$

ESEMPIO DERIVABILE



ESEMPIO NON DERIVABILE



$$\mathcal{E}_6: \frac{e \rightarrow e'}{e_0 \text{ bop } e \rightarrow e_0 \text{ bop } e'}$$

Per permettere la derivazione dobbiamo aggiungere una nuova regola ($\mathcal{E}_6$), ma questa regola rende il sistema di transizione non deterministico perché quando abbiamo un operatore applicato a due espressioni non costanti, abbiamo almeno due regole ($\mathcal{E}_2$ e $\mathcal{E}_6$) che possiamo applicare e quindi due transizioni possibili. Ci possono essere problemi quando, nel linguaggio, il valore risultato può dipendere dall'ordine di valutazione degli operandi, e questo non va bene, per questo nella semantica si tende a dare un ordine fissato. Quindi le regole $\mathcal{E}_1$ – $\mathcal{E}_3$ permettono solo la valutazione da sinistra verso destra. Mentre $\mathcal{E}_6$ permetterebbe una valutazione indipendente dei due sottoalberi, più coerente con i possibili modi di valutare

un'espressione, ma con maggiore possibilità di errore nell'implementazione. Quindi potremmo dare più regole ma non è necessario e può essere controproducente, anche se così perdiamo alcune proprietà matematiche, come la commutatività.

Osserviamo quindi come nella semantica tutto quello che specifichiamo è possibile, mentre quello che non si può derivare da quanto specificato non è possibile, viceversa tutto quello che è derivabile dalle regole deve essere desiderato (altrimenti abbiamo sbagliato la specifica). Non è invece possibile specificare cosa non vogliamo derivare.

LE REGOLE → (CONTINUA)

Continuano quindi ad introdurre le regole, ora vediamo quelle per le espressioni booleane, molto simili a quelle aritmetiche, con la sola aggiunta dell'operatore unario.

$$\mathcal{E}_4: k_1 \text{ bop } k_2 \rightarrow t \quad \text{se } k_1 \text{ bop } k_2 = t, \\ k_1, k_2 \in \mathcal{BUN}, t \in \mathcal{B}, \text{ bop} \in \{=, \text{or}\}$$

Anche questa regola è un assioma, che semplicemente valuta l'espressione sintattica contenente un operatore, nel valore che esso rappresenta, chiaro che se l'operatore è booleano i valori coinvolti devono essere entrambi booleani (di questa verifica si parlerà più avanti).

La regola successiva è esattamente la regola $\mathcal{E}_3$ dove ammettiamo che l'espressione contenga operatori binari booleani. Anche in questo caso fissiamo l'ordine di valutazione da sinistra verso destra.

$$\mathcal{E}_2: \frac{e \rightarrow e'}{e \text{ bop } e_0 \rightarrow e' \text{ bop } e_0}$$

La regola successiva stabilisce invece che, nel momento in cui l'operando a sinistra è un valore booleano allora posso iniziare a valutare l'operando a destra. Chiaro che, nuovamente, quando anche l'operatore a destra è un valore booleano allora ricadiamo nell'assioma $\mathcal{E}_4$ e possiamo restituire il valore finale.

$$\mathcal{E}_5: \frac{e \rightarrow e'}{t \text{ bop } e \rightarrow t \text{ bop } e'}$$

Nel caso booleano, dobbiamo aggiungere la regola per l'operatore unario. Per prima cosa l'assioma che restituisce il valore corrispondente all'applicazione dell'operatore sul valore rappresentato, e poi la regola di valutazione, analoga alle precedenti.

$$\mathcal{E}_6: \text{not } t_1 \rightarrow t \quad \text{se } \text{not } t_1 = t, t_1 \in \mathcal{B}$$

$$\mathcal{E}_7: \frac{e \rightarrow e'}{\text{not } e \rightarrow \text{not } e'}$$

VALUTAZIONE ED EQUIVALENZA

Come abbiamo più volte sottolineato, le regole della semantica dinamica, ovvero del sistema di transizione, descrivono formalmente come vengono valutate le espressioni del nostro linguaggio. Quindi, possiamo usare queste regole ora per definire formalmente cosa significa valutare una espressione.

Sia k una costante (valore intero o booleano). Formalmente k è il significato di un oggetto (simbolo) sintattico k dentro $\mathcal{NUB}$. Nel nostro caso quindi k è un valore dentro $\mathcal{NUB}$ (numeri naturali e valori booleani). Allora la valutazione è una funzione che prende una espressione sintattica da valutare, ovvero un elemento in $\mathcal{E}$ (una configurazione del sistema di transizione) e gli associa il valore che esso rappresenta (significato dato da una configurazione finale),

Valutazione delle espressioni:

La valutazione è una funzione $\text{Eval}: \mathcal{E} \rightarrow \mathcal{NUB}$ che descrive il comportamento dinamico delle espressioni restituendo il valore in cui esse sono valutate:

$$\text{Eval}(e) = k \Leftrightarrow e \rightarrow^* k$$

dove $\rightarrow^*$ è la chiusura transitiva della relazione di transizione $\rightarrow$.

A questo punto, il concetto di valutazione può essere usato per definire formalmente quando due espressioni sono equivalenti, ovvero quando due espressioni anche diverse sintatticamente, rappresentano invece lo stesso valore, ovvero hanno lo stesso *significato*.

Equivalenza di espressioni:

L'equivalenza di espressioni è una relazione $\equiv \subseteq \mathcal{E} \times \mathcal{E}$ definita come segue:

$$e_0 \equiv e_1 \Leftrightarrow \text{Eval}(e_0) = \text{Eval}(e_1)$$

Grazie a questa definizione, possiamo quindi dimostrare che, ad esempio, l'espressione $(3+5)*2$ è equivalente all'espressione $(1+3)*4$, pur essendo due espressioni sintatticamente molto diverse.